ASSIGNMENT 3 – CO3

Q1 – Constructors and Constructor Overloading (6 Marks)

1. Implementation: Library Book Records (Delegation & Validation)

```
class Book {
    std::string title, author;
    int copies;
public:
    Book(const std::string& t, const std::string& a, int c)
        : title(t), author(a), copies(c < 0 ? 0 : c) {
        if (c < 0) std::cout << "  [Guard] Negative copies rejected -> set to 0\n";
    }
    explicit Book(const std::string& t) : Book(t, "Anonymous Donor", 1) {} // delegates
    Book() : Book("Untitled", "Unknown", 0) {}                             // delegates
};
```

2. Design & Analysis

- **When the compiler supplies a default constructor:** Only when no user-declared constructors exist. Once any parameterised constructor is written, the compiler suppresses the implicit default constructor (confirmed when BookNoDef shelf[3]; failed with error: no matching function for call to 'BookNoDef::BookNoDef()').
- **Why member initializer lists are preferred:** They initialize members directly in one step, whereas assignment inside the constructor body first default-constructs members (e.g., std::string) and then overwrites them. Constants and references can only be initialized in the list.
- **Initialization order:** Members are initialized in their **declaration order in the class**, not the order written in the initializer list. Writing ReorderDemo(int x) : b(x), a(b + 1) {} when a is declared before b triggered warning: 'ReorderDemo::b' will be initialized after [-Wreorder] and [-Wuninitialized].
- **State of three objects after construction:** full → ("Clean Code", "Robert Martin", 5); donated → ("The C++ Programming Language", "Anonymous Donor", 1); empty → ("Untitled", "Unknown", 0); invalid copies = -3 was clamped to 0.

3. Design Question (explicit) & Alternative Concept Analysis

When to use explicit: Mark single-argument constructors explicit to block unintended implicit conversions. Without explicit, registerDonation(std::string("C++")) compiles by silently constructing a temporary Book; with explicit Book(const std::string& t), it fails at compile time.

Design	Time / Space	Validation Checks	Trade-off
Overloaded + Delegation	O(1) / O(1)	1 place (master ctor)	Clear distinct call signatures
Single Default-Arg Ctor	O(1) / O(1)	1 place (single ctor)	Shorter code, allows 2-arg calls

Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)

1. Shallow Copy Failure vs Rule of Three (Complete the Code)

Without a user-defined copy constructor, copying Prescription q = p; copies only the pointer medicines. Both objects point to the same heap array: modifying q mutates p, and when both destructors run, the second delete[] triggers a double free. Under -fsanitize=address, q2_shallow.cpp aborted with ERROR: AddressSanitizer: heap-use-after-free in ~Prescription().

```
Prescription(const Prescription& other) : n(other.n), medicines(new std::string[other.n]) {
    for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i]; // TODO 1
}
Prescription& operator=(const Prescription& other) {              // TODO 2
    if (this == &other) return *this; // self-assignment guard
    std::string* newArr = new std::string[other.n];
    for (int i = 0; i < other.n; ++i) newArr[i] = other.medicines[i];
    delete[] medicines; medicines = newArr; n = other.n;
    return *this;
}
~Prescription() { delete[] medicines; }                           // TODO 3
```

Observed ASan-Clean Output: Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS. Self-assignment p = p preserved Ibuprofen ORS with zero sanitizer errors.

2. Design, Experimental Investigation & Alternative Analysis

- **Memory Diagrams:** Shallow copy: p.medicines and q.medicines → same heap block [Paracetamol, ORS]. Deep copy: p.medicines → [Ibuprofen, ORS]; q.medicines → separate block [Paracetamol, Cetirizine].
- **Rule of Three & Call Sites:** A class managing a raw resource needs all three (destructor, copy constructor, copy assignment). Copy constructor runs on: (1) initialization Prescription q = p;, (2) pass-by-value (1 copy vs 0 for const&), and (3) return-by-value. Rely on the compiler default only when all members manage themselves (Rule of Zero, e.g., std::vector<std::string>).

Copy Design / Size	Time Complexity	Space Complexity	Measured Deep-Copy Time
Deep Copy (n = 10)	O(n)	O(n)	5.104 µs
Deep Copy (n = 1,000)	O(n)	O(n)	30.423 µs
Deep Copy (n = 100,000)	O(n)	O(n)	3356.65 µs
Shallow Copy / Rule of Zero	O(1) / O(n)	O(1) / O(n)	Shallow unsafe; vector safe

Q3 – Dynamic Constructors and Destructors (7 Marks)

1. Auditorium 2D Seat Map & RAII Analysis

The dynamic constructor allocates char** seatMap = new char*[rows] and seatMap[i] = new char[seatsPerRow] (handling rows == 0 safely without allocating). The destructor frees every row with delete[] seatMap[i] followed by delete[] seatMap.

- **Lifecycle Order:** Local stack objects destruct in reverse declaration order at function exit; block-scope objects destruct immediately at }; heap objects destruct only when delete is called. Forgetting delete leaks memory (caught by LeakSanitizer); using delete instead of delete[] on an array causes Undefined Behaviour.
- **Destructor Rules & RAII:** A destructor takes no arguments and cannot be overloaded because it is invoked automatically by the runtime with no call-site arguments. Tying cleanup to the destructor (RAII) guarantees release even on early returns.
- **Measured Seat Map Memory:** 10×20 → 280 B; 100×200 → 20,800 B; 1,000×2,000 → 2,008,000 B (ASan clean).

2. Predict the Output (Logger Struct)

> **Pre-Execution Prediction vs Observed Output**
> **Predicted:** +A +B -B copyAc use -Ac +P -P end -A
> **Observed:** +A +B -B copyAc use -Ac +P -P end -A (Exact match).

a is constructed first and destroyed last. b is destroyed at the inner block's closing brace. Passing a by value to use(Logger) invokes the copy constructor (copyAc) and destroys that parameter copy (-Ac) upon returning. p is created and destroyed explicitly via new/delete.

3. Alternative Concept & Complexity Analysis

2D Layout	Access Time	Heap Allocations	Extra Pointer Space
Array of Row Pointers	O(1)	rows + 1	rows × 8 B (8 KB for 1000 rows)
Flat 1D Block (r*seats+s)	O(1)	**1** allocation	0 bytes
std::vector<std::vector>	O(1)	rows + 1	rows × 24 B (vector headers)

Q4 – Overloading Arithmetic and Stream Operators (7 Marks)

1. Complete the Code & Experimental Investigation

```
Money operator+(const Money& rhs) const { return Money(0, 0) += ...; } // returns new Money
bool operator==(const Money& rhs) const { return paise == rhs.paise; }
friend std::ostream& operator<<(std::ostream& os, const Money& m);
```

- **Observed Results:** a(120,75) + b(79,25) → Rs 200.00, true. Carry test Rs 0.75 + Rs 0.50 → Rs 1.25. Negative subtraction 0.50 - 0.75 → -Rs 0.25 (handled by printing a leading minus sign before absolute rupees/paise).
- **1,000,000 Additions vs double:** Storing integer paise in long long avoids binary floating-point rounding drift (Money gave exact Rs 10000.00; double accumulates representation error for fractions like 0.1/0.01).
- **Design Rules:** + returns a new Money value (operands must not change), whereas += mutates *this and returns Money& for chaining. << must be a non-member friend because its left operand is

std::ostream&, not Money. The compiler translates a + b into a.operator+(b) (or operator+(a, b) for a friend).

- **Engineering Decision:** Multiplying two Money objects (Money * Money) makes no dimensional sense (rupees²) and must be disallowed; scalar scaling (Money * int) is valid.

2. Alternative Concept & Complexity Analysis

All designs (member operators, non-member friend operators, or named functions add()/equals()) execute in **O(1)** time and **O(1)** space. Making operator+ a **non-member friend** allows implicit conversion on both operands, enabling both m + 100 and 100 + m (whereas a member function only allows m + 100).

Q5 – Overloading Increment, Comparison and Subscript Operators (6 Marks)

1. Design, Investigation & Predict the Output

- **Prefix vs Postfix ++:** The compiler distinguishes postfix operator++(int) via a dummy int parameter. Postfix must save and return the old copy, making it slower than prefix (which increments in place and returns *this by reference).
- **operator[] Const & Non-Const:** Non-const std::string& operator[](int) allows modifying pl[0] = "Track 1"; const std::string& operator[](int) const allows reading a const Playlist. Out-of-range indices (pl[-1], pl[3]) print a range error and return a safe dummy string.
- **Roll-over & Predict (Counter):** Advancing 0:59:59 via a = t++ and b = ++t gave a = 0:59:59, b = 1:00:01, t = 1:00:01. For Counter c{5}; a = c++; b = ++c;, predicted and observed output is 5 7 7.

10,000,000-Call Benchmark	-O0 Time	-O2 Time	Takeaway
Prefix ++t vs Postfix t++	28.52 ms / 62.98 ms	10.35 ms / 9.43 ms	++i avoids temp copy
Checked [] vs Unchecked	44.18 ms / 35.83 ms	20.01 ms / 9.70 ms	Bounds check adds ~10 ms at -O2

Conclusion: Programmers prefer ++i over i++ for class types because it avoids constructing and destroying a temporary copy when the old value is not needed.

Q6 – The Rules of Operator Overloading (7 Marks)

1. Rules & Experimental Investigation (Text Class)

- **Non-overloadable operators:** :: (scope resolution), . (member access), .* (pointer-to-member), ?: (ternary), and sizeof cannot be overloaded because they operate on names, types, or compile-time layout rather than runtime values.
- **Fundamental Rules:** Precedence, associativity, and arity cannot change; at least one operand must be a user-defined type. operator= must be a member function so the compiler never conflicts with its synthesized assignment operator. Binary symmetric operators (+, <<) should be friends.
- **ASan & Copy Elision:** Under -fsanitize=address, Text passed all tests (empty string, long SMS, self-assignment msg = msg) with zero leaks. Evaluating a + b + c triggered **0 copy constructor calls** in C++17 due to guaranteed copy elision.

2. Alternative Concept & 100,000-Concatenation Benchmark

Evaluating a + b + c for lengths n1, n2, n3 copies (n1 + n2) + (n1 + n2 + n3) characters (2 heap allocations). In a loop of 100,000 concatenations, repeated tAcc = tAcc + tUnit is **O(N²)** and took **12,279.7 ms**, whereas std::string::operator+= uses geometric buffer growth (**O(N)** amortized) and finished in **1.36 ms** (>9,000x faster).

Q7 – Design a Complex-Number Class from First Principles (10 Marks)

Stage 1 – Interpret the Problem

- **Data & Operations:** Store double re, im. Provide default Complex() (0+0j), real-only Complex(double r), and Complex(double r, double i) constructors; support +, -, *, /, ==, <<, and magnitude().
- **Measurable Requirements:** Compute series/parallel RLC impedances to ≥4 decimal places, guard against division by zero ($|Z|^2 < 10^{-12}$), and run all arithmetic operations in O(1) time.

Stage 2 – Class Design & Formulas

All arithmetic and comparison operators are friend functions (symmetric operands, enabling double + Complex), while constructors and magnitude() are members:

- $(a + bj) \pm (c + dj) = (a \pm c) + (b \pm d)j$
- $(a + bj) \times (c + dj) = (ac - bd) + (ad + bc)j$
- $(a + bj) / (c + dj) = [(ac + bd) + (bc - ad)j] / (c^2 + d^2)$ (returns 0 + 0j with error message if denominator < 10^{-12})

Stage 3 – Implement & Verify (RLC Circuit at ω = 1000 rad/s)

For R = 100 Ω (Z_R = 100 + 0j), L = 0.2 H (Z_L = 0 + 200j), and C = 10 μF (Z_C = 0 - 100j):

Circuit Configuration	Hand Calculation	Observed Program Output
Series RLC ($Z_R + Z_L + Z_C$)	100 + 100j, $\|Z\| = \sqrt{(100^2+100^2)}$ = 141.4214 Ω	100.0000 + 100.0000j | 141.4214 ohm
Parallel ($Z_R \parallel (Z_L+Z_C)$)	(100×100j)/(100+100j) = 50 + 50j, |Z| = 70.7107 Ω	50.0000 + 50.0000j | 70.7107 ohm
Division by Zero (Z_R / 0)	Denominator = 0 → error handled safely	[Error] Division by zero! -> 0.0000 + 0.0000j

Stage 4 – Defend the Design & Alternative Analysis

Compared to std::complex<double>, both use **O(1)** time and **16 bytes** (two doubles) per object. std::complex handles extreme overflow/underflow scaling in division (Smith's algorithm) and produces NaN/inf on zero division per IEEE 754, whereas our custom Complex prints rectangular a + bj format directly and enforces domain-specific zero-impedance error messages. One remaining limitation is that naive $c^2 + d^2$ can overflow for magnitudes > 10^{154}.